LOCAL SIMULATION PROTOTYPE | RELEASE V2.0.0 · PHASES M0 → M2 | 110 / 110 AUTOMATED TESTS PASSING

CLEANPOINT TECHNICAL MANUAL
End-to-End System Architecture, Module Reference & Operational Runbook

CleanPoint: End-to-End Working Architecture & Module Reference

Complete engineering documentation for the local, defensively designed ransomware-recovery analysis simulator: content-addressed snapshot storage, explainable **rules-v1** anomaly detection, conservative recovery point selection with abstention, isolated verified restore, deterministic 8-family Python scenario engine, and Phase M2 React/TypeScript operator console.

CORE ARCHITECTURAL PRINCIPLE — SAFETY BEFORE RECENCY

After a destructive incident, the newest backup often already holds encrypted or corrupted bytes. A valid SHA-256 checksum proves **storage consistency**, never **absence of compromise**. CleanPoint evaluates behavioral telemetry + manifest content structure, recommends the newest strictly pre-onset clean snapshot, or explicitly abstains (`no_safe_candidate`) rather than silently falling back to a poisoned backup.

0 / 14	100%	8 Families	15 Views
Unsafe snapshot selections on locked corpus (release target: 0)	Known-attack recall (8/8) & 14/14 verified isolated restores	Deterministic seeded Python scenarios + air-gapped ground truth	Embedded Phase M2 React + TypeScript operator web console

Backend Core: Go 1.26.1 · Pure-Go SQLite WAL (`modernc.org/sqlite`)

Simulator & Harness: Python 3.11+ (100% Standard Library)

Web Console & Landing: React 18 · TypeScript 5.6 · Vite 5 · `go:embed`

API Contract: OpenAPI 3.0.3 (`/api/v1`, 21 endpoints, RBAC tokens)

Repository: `github.com/Bilalyzr/CleanPoint` (Commit `4838147`)

Safety Boundary: Synthetic bytes only · Loopback-bound · No host rollback

00 Document Map & Executive Overview

This manual documents the complete working implementation of **CleanPoint** from entrypoints to disk bytes. Every diagram, formula, SQL table, API route, UI module, and CLI command corresponds directly to verified source code in the repository.

01 · System Architecture & End-to-End Flow
Layered topology, trust boundaries, and 7-stage operational pipeline

02 · Complete Module-by-Module Reference
All Go binaries, 11 internal packages, web assets, and Python modules

03 · Data Model, SQLite Schema & Lifecycles
All 9 WAL tables, foreign-key constraints, object fan-out, state machines

04 · Detection (rules-v1) & Selector Math
300s windows, 7 features, Median/MAD baseline, 4-gate exposure, 5-gate selector

05 · Deterministic Simulator & Evaluation Proof
8 scenario families, 60/20/20 seed split, locked-corpus & latency benchmarks

06 · REST API Contract & M2 Operator Console
All 21 HTTP endpoints, error codes, RBAC roles, and 15 console views

07 · Security Architecture, Threat Model & Tests
Path/symlink defenses, logical immutability, limitations, 110-case matrix

08 · End-to-End Working Runbook
Windows PowerShell, Linux/macOS Bash, CLI commands, and Docker workflow

Honest Positioning & Safety Posture (Mandatory Disclosure)
CleanPoint is a **local defensive ransomware-recovery analysis prototype** built for cyber-resilience engineering and analysis. **No live malware** is ever executed, packaged, or analyzed — "encrypted" file contents are deterministic seeded pseudo-random bytes. It binds to loopback (`127.0.0.1`) by default, uses API-level logical immutability (not hardware WORM), and restores exclusively into isolated `sandbox/restored/` directories.

01 Complete System Architecture & End-to-End Flow

CleanPoint runs as a **single self-contained Go HTTP server process** backed by a local transactional SQLite database (`WAL` mode, `foreign_keys = ON`) and a restricted filesystem sandbox, paired with a deterministic zero-dependency Python 3 scenario generator and evaluation harness. Both the public landing page (`/`) and the compiled React M2 console (`/app/`) are embedded into the Go binary at compile time via `go:embed` .

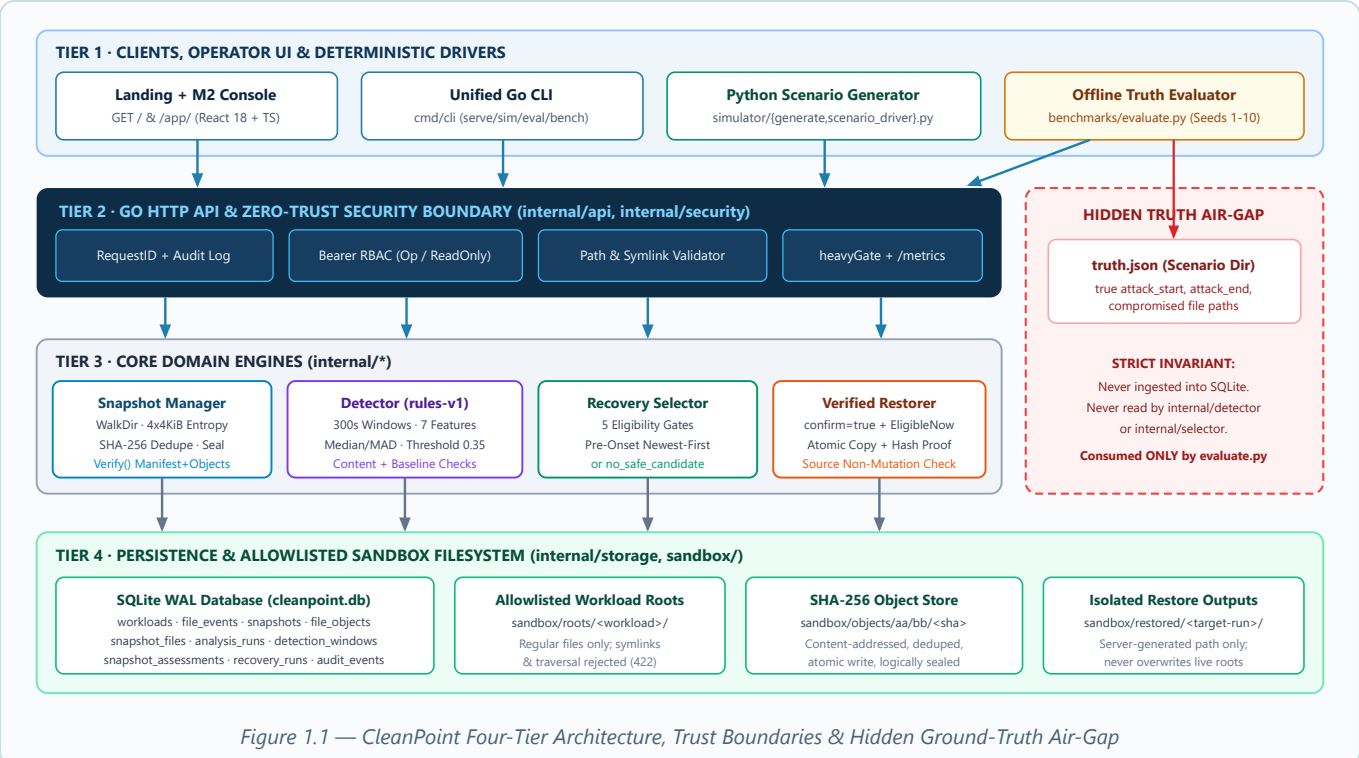


Figure 1.1 — CleanPoint Four-Tier Architecture, Trust Boundaries & Hidden Ground-Truth Air-Gap

1.2 End-to-End Operational Pipeline (7-Stage Execution Flow)

Every scenario run (whether driven via CLI `cleanpoint simulate`, `cleanpoint demo`, `benchmarks/evaluate.py`, or the M2 Console's **Simulation Lab**) executes the same deterministic seven-stage pipeline:

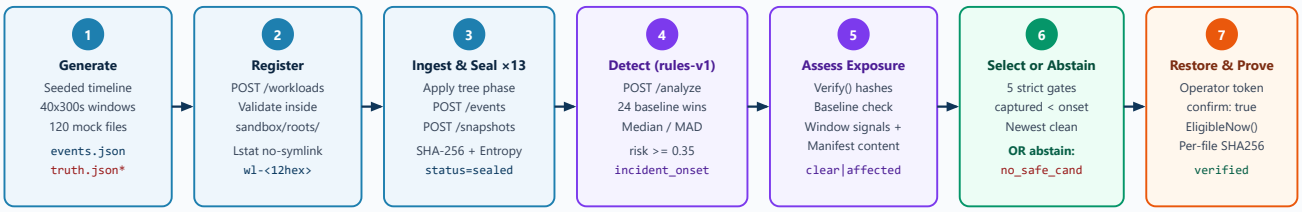


*Figure 1.2 — End-to-End Seven-Stage Scenario, Detection, Selection & Verified Recovery Pipeline (*truth.json stays local)*

Stage-by-Stage Execution & Data-Flow Walkthrough

Pipeline Stage	Executing Module	SQLite & Sandbox Disk Mutations	Key Invariants & Failure Guards
1. Generate	`simulator/generate.py` `simulator/scenarios.py`	Writes `events.json`, `phases/phase_00..12/`, and local-only `truth.json` into scenario workdir.	Deterministic seeded PRNG (`random.Random`); identical output across Windows, Linux, and macOS.
2. Register	`POST /api/v1/workloads` `internal/security`	Inserts row in `workloads` (`wl-*`) + `audit_events`; validates directory under `sandbox/roots/`.	`ValidateRootInside` + `CheckNoSymlinks` reject traversal, external roots, or symlinks (`422`).
3. Ingest & Seal (×13)	`POST .../events` `POST .../snapshots` `internal/snapshots`	Reconstructs tree at windows `3,6..39`; inserts `file_events`; writes `objects/aa/bb/<sha>`; seals `snapshots` + `snapshot_files`.	Idempotent `INSERT OR IGNORE` by event ID; `captured_at` anchored to latest event clock; rejects >5,000 files or >100 MiB (`413`).
4. Detect (rules-v1)	`POST .../analyze` `internal/detector`	Creates `analysis_runs` (`ar-*`) and 40 `detection_windows` rows with feature & explanation JSON.	First 24 windows calibrate Median/MAD; windows ≥24 scored against `0.35` threshold; zero access to `truth.json`.
5. Assess Exposure	`snapshots.Verify()` `detector.AssessSnapshot`	Re-hashes every referenced object on disk; upserts `snapshot_assessments` without mutating `snapshots`.	Checks `BaselineContaminated`, 24-window coverage, manifest content (`≥30%` novel ext @ `H≥7.0`), and window overlap.
6. Select or Abstain	`GET .../recovery-candidates` `internal/selector`	Read-only evaluation over `snapshots` + `snapshot_assessments` + latest `onset_time`; increments abstention metric if 0 qualify.	Enforces all 5 gates; sorts newest-first; returns `no_safe_candidate` with reasons if every backup is tainted or unverified.
7. Restore & Prove	`POST .../restore` `internal/restore`	Creates `recovery_runs` (`run-*`); atomically materializes files into `sandbox/restored/<label>-<hex>/`; logs audit trail.	Requires Operator token + `confirm:true` + live `EligibleNow()`; verifies output SHA-256 and post-copy `source_unchanged=true`.

02 Complete Module-by-Module Reference

CleanPoint is organized into strict single-responsibility packages across four layers: Go binary entrypoints (`cmd/`), core Go domain packages (`internal/` & `migrations/`), embedded web applications (`web/`), and deterministic Python simulation/evaluation suites (`simulator/` & `benchmarks/`).

2.1 Repository Directory Layout

```
CleanPoint/
├── cmd/
│   ├── server/main.go              # Standalone local API + embedded web UI server (cleanpoint-server.exe)
│   ├── cli/{main.go,serve.go}      # Unified CLI: serve, simulate, evaluate, benchmark, demo (cleanpoint.exe)
│   └── simulator/main.go           # Go wrapper launching simulator/generate.py (cleanpoint-simulator.exe)
├── internal/
│   ├── config/config.go            # CLEANPOINT_* env loader, sandbox bootstrap, CSPRNG .tokens manager
│   ├── domain/models.go            # Canonical entities, UTC millisecond timestamps, state constants, IDs
│   ├── security/paths.go           # Path traversal/device/backslash/symlink guards & atomic file writes
│   ├── storage/{db,store,repos,json}.go # Pure-Go SQLite WAL connection, migrations, transactional CRUD
│   ├── snapshots/manager.go        # Content-addressed SHA-256 object store, 4x4KiB entropy, sealing, Verify()
│   ├── detector/{features,baseline,detector,analyze}.go # 300s windows, Median/MAD, rules-v1, exposure
assessment
│   ├── selector/selector.go        # 5-gate conservative candidate ranking, abstention, EligibleNow()
│   ├── restore/restore.go          # Operator-confirmed isolated restore, per-file hash proof, source re-
verify
│   ├── metrics/metrics.go          # Lock-protected Prometheus text counters (/api/v1/metrics)
│   └── api/{server,middleware,handlers,analysis,m2,webui,errors}.go # HTTP mux, RBAC, M2 endpoints, SPA handler
├── migrations/{001_init.sql,migrations.go} # Embedded SQLite DDL (9 tables, WAL, foreign_keys=ON)
├── web/
│   ├── embed.go                    # go:embed landing.{html,css,js} + all:console-dist
│   ├── landing.{html,css,js}       # Public scroll-driven landing page + 3D canvas snapshot corridor
│   ├── console/                    # Phase M2 React 18 + TypeScript + Vite source (15 PRD views)
│   └── console-dist/               # Prebuilt production SPA bundle served at /app/
├── simulator/{scenarios,generate,scenario_driver,demo}.py + seeds.json
├── benchmarks/{evaluate,baselines,latency}.py + throughput.sh
├── tests/{harness,security,recovery,integration}/ # 110 automated test cases across 10 suites
├── api/openapi.yaml                # Complete OpenAPI 3.0.3 REST contract
├── scripts/{bootstrap,test,demo,benchmark}.sh + {build,test,demo,benchmark}.ps1
└── Dockerfile · compose.yaml · Makefile · go.mod · requirements.txt · results/ · docs/
```

2.2 Go Binary Entrypoints (`cmd/`)

Module / Binary	Source Files	Responsibilities & Key Mechanics
Server `bin/cleanpoint-server.exe`	`cmd/server/main.go`	Loads `config.Load()` , opens SQLite via `storage.Open()` , mounts `api.NewServer()` with `ReadHeaderTimeout: 10s` , binds loopback (`127.0.0.1:8080` default or `CLEANPOINT_ADDR`), and handles graceful 5-second `SIGINT` / `SIGTERM` shutdown.
Unified CLI `bin/cleanpoint.exe`	`cmd/cli/main.go` `cmd/cli/serve.go`	Single binary supporting five subcommands: `serve` (in-process server with embedded UI), `simulate` , `evaluate` (`--scenario` , `--seeds` , `--json` , `--no-restore` , `--output`), native Go `benchmark snapshots|restore` (computes p50/p95/mean, MB/s, files/s), and `demo` . Includes upward `repoRoot()` discovery so invocation works from any working directory.
Simulator Launcher `bin/cleanpoint-simulator.exe`	`cmd/simulator/main.go`	Thin native wrapper resolving the repository root from executable/working directory and invoking `simulator/generate.py` via `CLEANPOINT_PYTHON` (default `python`) so Python remains the single deterministic fixture source of truth.

2.3 Core Go Internal Packages (`internal/` & `migrations/`)

1. `internal/config` — BOOTSTRAP

Loads runtime configuration from `CLEANPOINT_*` environment variables with safe local-only defaults.

- Creates `sandbox/{roots,objects,restored}` automatically.
- Generates 24-byte CSPRNG hex tokens for `operator` and `readonly` roles into `sandbox/.tokens` (`0600`) if unset; reuses existing tokens across restarts; **never logs tokens**.
- Enforces MVP caps: `MaxFiles=5000` , `MaxBytes=100 MiB` , `WindowSeconds=300` , `BaselineWindows=24` , `RiskThreshold=0.35` .

2. `internal/domain` — ENTITIES

Shared domain structs, state machine vocabularies, and canonical timestamp/ID primitives.

- Fixed-width UTC format `2006-01-02T15:04:05.000Z` ensures SQLite string sorting matches chronological order.
- Defines lifecycle enums for Snapshots (`creating|sealed|invalid`), Exposure (`unknown|clear|suspect|affected`), Alerts (`normal|suspicious`), and Restores (`requested|verifying|verified|failed`).
- Generates random 12-hex-char prefixed IDs (`wl-` , `snap-` , `ar-` , `win-` , `run-`).

3. `internal/security` — ZERO-TRUST FS

Centralized filesystem boundary enforcement used on every untrusted path input and disk operation.

- `ValidateRelativePath` : rejects empty strings, NUL bytes, backslashes (`\`), leading `/` , drive letters (`C:`), `.` / `..` , leading/trailing spaces, and DOS devices (`CON` , `PRN` , `AUX` , `NUL` , `COM1..9` , `LPT1..9`).
- `ValidateRootInside` & `CheckNoSymlinks` : walks every path segment with `os.Lstat` to block symlink escapes.
- `AtomicWrite` : writes same-directory `.tmp-*` , calls `Sync()` , `Chmod()` , and atomic `os.Rename()` .

4. `internal/storage` + `migrations` — SQLITE WAL

Pure-Go SQLite persistence (`modernc.org/sqlite` , zero CGO) with embedded schema migrations.

- Configures `PRAGMA journal_mode=WAL` , `foreign_keys=ON` , `busy_timeout=5000` , single-writer pool (`SetMaxOpenConns(1)`) to prevent `SQLITE_BUSY` races.
- `IngestEvent` uses `INSERT OR IGNORE` by event ID for strict idempotency (returns `ingested` vs `duplicates`).
- Separates immutable `snapshots` / `snapshot_files` from mutable `snapshot_assessments` .

5. `internal/snapshots` — OBJECT STORE

Content-addressed snapshot capture, canonical manifest hashing, logical sealing, and deep integrity verification.

- `Capture` defaults `captured_at` to the workload's latest ingested event timestamp (the simulation clock); refuses capture if zero telemetry exists.
- Walks regular files, computes SHA-256 + 4×4 KiB Shannon entropy, deduplicates into `objects/aa/bb/<sha256>` , and seals canonical JSONL `ManifestHash` .
- `Verify` recomputes manifest hash from SQL rows and re-hashes every on-disk object byte-for-byte.

6. `internal/detector` — RULES-V1

Explainable behavioral + manifest-content anomaly engine operating strictly without ground truth.

- Buckets events into epoch-aligned 300s windows (including quiet empty windows); replays file-existence counts at every boundary.
- Calibrates Median and `1.4826 × MAD` scale across the first 24 baseline windows; scores later windows with 6 weighted signals (threshold `0.35`).
- Combines window risk, baseline-contamination check, and manifest content check (`≥30%` novel extensions at `≥7.0` entropy) to classify snapshots.

7. `internal/selector` — DECISION GATE

Ranks snapshots newest-first under five mandatory safety gates or abstains with explicit reasons.

- Requires: (1) `sealed` , (2) `integrity_ok` , (3) `coverage_ok` , (4) `exposure == clear` , and (5) `captured_at < incident_onset` .
- Deterministic tie-break: newer `captured_at` → higher `file_count` → larger `snapshot_id` .
- `EligibleNow` performs a fresh live re-check (re-hashing every object) immediately before any restore executes.

8. `internal/restore` — VERIFIED COPY

Executes operator-confirmed, isolated restores with dual pre- and post-copy cryptographic verification.

- Requires `confirm == true` , valid `SafeTargetName` (`^[a-z0-9][a-z0-9-]{0,63}$`), and passing `EligibleNow()` .
- Writes exclusively into server-generated `sandbox/restored/<target>-<runhex>/` .
- Hashes source object → atomic temp write + rename → re-reads & re-hashes written file → re-verifies source snapshot to prove `source_unchanged = true` .

9. `internal/api` + `internal/metrics` HTTP & RBAC

Go 1.22+ pattern-routed `http.ServeMux` serving REST endpoints, Prometheus metrics, landing page, and SPA.

- `WithRequestID` assigns `X-Request-Id` (`req-<16hex>`) and logs status + latency without leaking headers.
- `Authenticate` uses constant-time `subtle.ConstantTimeCompare` ; `RequireOperator` blocks read-only mutations (`403`) and writes audit trail entries.
- `heavyGate` mutex serializes concurrent snapshot captures & restores (`409 busy`).
- `m2.go` powers snapshot manifest inspection, analysis history, measured evaluation serving, and single-concurrency Simulation Lab background jobs (`simRegistry`).

10. `web/` + `simulator/` + `benchmarks/` UI & CORPUS

Embedded web frontends and deterministic Python 3 scenario/benchmark engines.

- `web/landing.*` : Public scroll story + zero-dependency 2D-canvas 3D snapshot timeline corridor at `GET /` .
- `web/console/` : React 18 + TypeScript + Vite console compiled to `web/console-dist/` and served at `GET /app/` with SPA deep-link fallback.
- `simulator/` : 8 deterministic scenario families (`scenarios.py`), phase reconstructor (`generate.py`), API driver (`scenario_driver.py`), and 3-scene narrative (`demo.py`).
- `benchmarks/` : 3-selector comparison harness (`evaluate.py`), latency probe (`latency.py`), and concurrency probe (`throughput.sh`).

2.4 Web Frontend, Simulator & Benchmark File-by-File Reference

File Path	Runtime Layer	Detailed Module Responsibility
`web/embed.go` `web/landing.{html,css,js}`	Go `embed.FS` + Vanilla Canvas	Embeds public landing page and `console-dist` into server binary; renders scroll-synced 3D snapshot timeline corridor with `prefers-reduced-motion` and tab-visibility pause support.
`web/console/src/App.tsx` `auth.tsx` · `api.ts` · `types.ts`	React 18 + TypeScript 5.6	Hash-routed SPA shell (`#/` , `#/workloads` , `#/simulation` , `#/evaluations` , `#/audit` , `#/settings`), typed REST client attaching `Authorization: Bearer` , and session role detection.
`web/console/src/components/*` `web/console/src/pages/*`	React 18 + SVG Viz	Custom zero-chart-library interactive SVGs (`RiskChart.tsx` , `SnapshotTimeline.tsx`), `RestoreDialog.tsx` operator safeguard modal, `Badges.tsx` semantic exposure chips, and 7 page views.
`simulator/scenarios.py` `simulator/generate.py`	Python 3.11+ Stdlib	Defines the 8 deterministic scenario families across 40×300s windows; generates initial 120-file workspace, phase snapshots, exact 4-chunk Shannon entropies, `events.json` , and hidden `truth.json` .
`simulator/scenario_driver.py` `simulator/demo.py` · `seeds.json`	Python 3.11+ Stdlib	Drives a generated scenario against a live CleanPoint HTTP server (register → 13× ingest+capture → analyze → select → optional restore); runs 3-scene guided terminal demo; locks 60/20/20 seed splits.
`benchmarks/evaluate.py` `baselines.py` · `latency.py`	Python 3.11+ Stdlib	Compares CleanPoint against naive `newest` and `integrity_only` baselines using `truth.json` ; asserts `unsafe_selections == 0` ; measures p50/p95 API latency and concurrent throughput.

03 Data Model, SQLite Schema & Entity Lifecycles

All persistent metadata lives in `migrations/001_init.sql` under SQLite `WAL` journaling with `PRAGMA foreign_keys = ON` and `ON DELETE RESTRICT` across every relation. Sealed snapshot records and content-addressed file objects are never mutated; analytical judgments live in `snapshot_assessments` so re-running an analysis with updated parameters never alters sealed evidence.

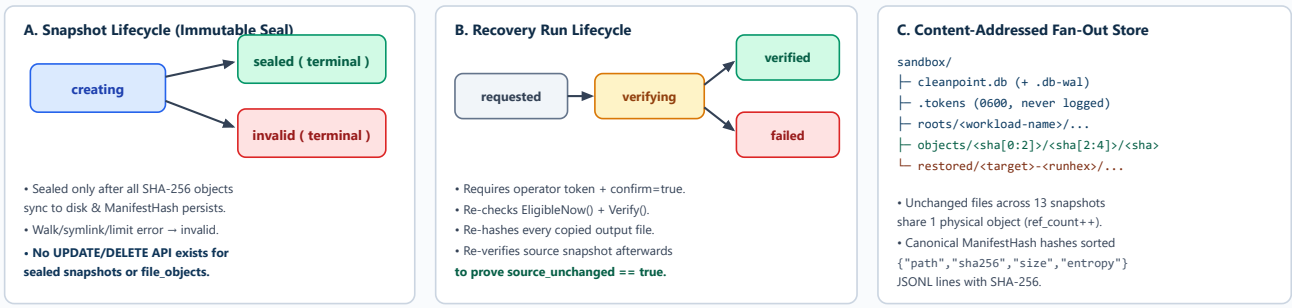


Figure 3.1 — Snapshot & Restore State Machines and Content-Addressed Sandbox Directory Topology

3.2 Complete SQLite Relational Schema (`migrations/001_init.sql`)

Table	Primary / Unique Keys	Foreign Keys & Indexes	Columns & Architectural Role
`workloads`	`PK(id)` `UNIQUE(name)`	Referenced by events, snapshots, analysis & recovery runs	`id` (`wl-*`), `name` (`[a-z0-9-]`), `root_path` (allowlisted under `sandbox/roots/`), `created_at` .
`file_events`	`PK(id)`	`FK(workload_id)` `idx_file_events_wl_time`	`event_time` , `operation` (`create|modify|delete|rename`), `path` , `dest_path` , `old/new_sha256` , `old/new_entropy` , `old/new_size` , `actor` , `scenario_id` , `ingested_at` . Idempotent by `id` .
`snapshots`	`PK(id)`	`FK(workload_id)` `idx_snapshots_wl_time`	`id` (`snap-*`), `captured_at` (simulation clock), `status` (`creating|sealed|invalid`), `manifest_hash` , `file_count` , `total_bytes` , `created_at` .
`file_objects`	`PK(sha256)`	Referenced by `snapshot_files`	`sha256` (lowercase 64-hex), `size_bytes` , `storage_path` (2-level fan-out), `ref_count` , `created_at` . Deduplicates identical file contents across snapshots.
`snapshot_files`	`PK(snapshot_id, relative_path)`	`FK(snapshot_id)` `FK(sha256)`	`relative_path` (validated slash-relative), `sha256` , `size_bytes` , `entropy` (rounded to 2 decimals). Forms the sealed canonical manifest.

Table	Primary / Unique Keys	Foreign Keys & Indexes	Columns & Architectural Role
`analysis_runs`	`PK(id)`	`FK(workload_id)` `idx_analysis_runs_wl`	`id` (`ar-*`), `detector_version` (`rules-v1`), `baseline_period` , `window_seconds` , `baseline_window_count` , `risk_threshold` , `status` , `onset_time` , `windows_evaluated` , `started_at` , `finished_at` .
`detection_windows`	`PK(id)` `UNIQUE(analysis_run_id, starts_at)`	`FK(analysis_run_id)` `FK(workload_id)`	`starts_at` , `ends_at` , `feature_json` , `risk_score` , `alert_level` (`normal|suspicious`), `explanation_json` (per-feature contributions), `is_baseline` , `event_count` .
`snapshot_assessments`	`PK(snapshot_id)`	`FK(snapshot_id)` `FK(analysis_run_id)`	`integrity_ok` , `coverage_ok` , `exposure_status` (`unknown|clear|suspect|affected`), `risk_score` , `affected_file_count` , `explanation_json` , `evaluated_at` . Upserted on each analysis run.
`recovery_runs`	`PK(id)`	`FK(workload_id)` `FK(snapshot_id)`	`id` (`run-*`), `status` , `target_name` , `output_path` , `total_files` , `verified_files` , `failed_files` , `source_unchanged` , `created_at` , `finished_at` , `report_json` (per-file outcomes).
`audit_events`	`PK AUTOINCREMENT(id)`	Chronological append-only trail	`actor` (`operator` / `read-only` / `anonymous`), `action` , `target_id` , `outcome` , `request_id` , `created_at` . Never stores bearer tokens.

04 Detection Engine (`rules-v1`) & Conservative Selector Design

CleanPoint purposefully uses an **explainable, deterministic statistical detector** (`rules-v1`) rather than an opaque black-box model. Every alert decomposes into exact per-feature values, robust baseline centers, z-scores, and bounded signal weights so an operator can audit *why* any snapshot was excluded.

4.1 Windowing & 4-Chunk Shannon Entropy Contract

- **Epoch-Aligned Non-Overlapping Windows:** Ingested events are bucketed into fixed `W = 300s` (5-minute) intervals anchored to Unix-epoch multiples of `300s` via `t.Truncate(300s)`. Empty quiet windows between active windows are preserved because quiet office intervals are genuine normal-baseline evidence.
- **Identical Go & Python Shannon Entropy Sampling:** Both `snapshots.EntropyBytes()` in Go and `generate.shannon_entropy()` in Python sample up to **four 4 KiB chunks** at offsets `{0, size/4, size/2, size - 4096}` (or all bytes when `size ≤ 16 KiB`) and compute `H = -Σ p(b) log2 p(b) ∈ [0, 8]` bits/byte.

4.2 Per-Window Feature Vector (7 Features)

Feature Name	Weight	ε Floor	Mathematical Definition & Missing-Evidence Policy
`modified_file_ratio`	**0.25**	0.01	Distinct paths touched by `create`, `modify`, or `rename` destination ÷ `max(1, filesAtWindowStart)` (derived by replaying prior create/delete/rename state). Always present.
`entropy_shift_ratio`	**0.25**	0.01	Eligible `create` / `modify` events with `new_entropy - old_entropy ≥ 1.5` bits/byte ÷ eligible events (requires both entropies present and `new_size ≥ 512` bytes). Marked **missing** if 0 eligible events.
`rename_rate`	**0.20**	0.50	`rename` events in window ÷ window duration in minutes (`5.0` min). Captures extension-churn ransomware behavior without relying on static extension blacklists.
`extension_novelty`	**0.15**	0.01	Fraction of distinct written file extensions in the window that were **never observed** across any of the 24 baseline windows (two-pass calculation via `ApplyExtensionNovelty`).
`delete_rate`	**0.10**	0.50	`delete` events in window ÷ window duration in minutes (`5.0` min). Captures shadow/original file wiping bursts.
`size_change_ratio`	**0.05**	0.01	`modify` events with `|new_size - old_size| / old_size ≥ 0.50` ÷ modify events with known `old_size > 0` and `new_size`. Marked **missing** if 0 eligible modifies.
`write_concentration`	0.00	0.01	Informational diagnostic: writes by busiest known `actor` ÷ total writes with a known actor. Recorded in window features for operator inspection; unscored in `rules-v1`.

4.3 Robust Median/MAD Baseline & `rules-v1` Risk Formula

The first `B = 24` windows (2 simulated hours) form the assumed-normal training baseline. Using median and Median Absolute Deviation (MAD) prevents isolated benign office spikes during training from inflating variance and blinding the detector:

```
center_i = median( { x_{w,i} : w ∈ [0 .. B-1], feature i not missing } )
MAD_i = median( { |x_{w,i} - center_i| } )
scale_i = max( 1.4826 × MAD_i , ε_i )

z_i = ( current_i - center_i ) / scale_i
signal_i = min( 1.0 , max( 0.0 , z_i ) / 5.0 ) ← saturates at z_i ≥ 5σ-equivalent
risk = Σ_{i ∈ Present} ( weight_i × signal_i ) ∈ [0.0, 1.0] (suspicious when risk ≥ 0.35)
```

Missing-Data Honesty: Features lacking telemetry evidence are flagged `Missing = true`, contribute `0` to `risk`, and reduce window `coverage = |Present| / 6`. Weights are **never** silently redistributed onto remaining features. `incident_onset` is defined as `StartsAt` of the earliest evaluated window (`w ≥ B`) with `risk ≥ 0.35`.

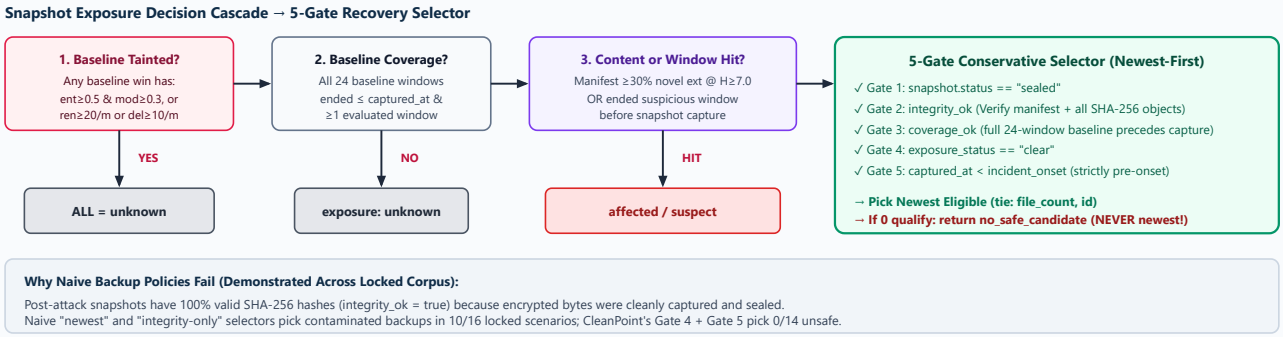


Figure 4.1 — Priority-Ordered Snapshot Exposure Classification & Five-Gate Conservative Selector

4.4 Priority-Ordered Snapshot Exposure Classifier (`internal/detector/detector.go`)

Priority	Condition Evaluated	Resulting Status	Engineering Rationale
Rule 1	`BaselineContaminated(baseWins)`: `ent_shift ≥ 0.50 && mod_ratio ≥ 0.30`, or `rename_rate ≥ 20/m`, or `delete_rate ≥ 10/m` in any baseline window	**UNKNOWN** (`coverage_ok=false`)	If destructive activity already occurred inside windows `0..23` (e.g., `pre_existing_contamination`), the Median/MAD baseline cannot certify any snapshot as clean.
Rule 2	Fewer than 24 baseline windows ended at `≤ captured_at`, or 0 evaluated post-baseline windows exist	**UNKNOWN** (`coverage_ok=false`)	Prevents recommending early baseline-period snapshots captured before the detector has observed a full 24-window calibration interval and post-baseline check.
Rule 3	`ManifestLooksCompromised` (`≥30%` of files carry non-baseline extensions & min 3 files at median entropy `≥ 7.0`) OR any suspicious window ended `≤ captured_at`	**AFFECTED**	Dual behavioral + content defense: catches both window-flagged attacks and snapshots whose on-disk manifest exhibits mass high-entropy novel-extension replacement.
Rule 4	A suspicious window overlaps `captured_at` (`StartsAt ≤ captured_at < EndsAt`) or pre-capture max risk `≥ 0.245` (`0.7 × 0.35`)	**SUSPECT**	Conservative buffer around in-flight or borderline pre-threshold activity; excluded from automatic clean eligibility.
Rule 5	Full 24-window uncontaminated baseline + 0 suspicious windows + clean manifest structure	**CLEAR**	Certified behavioral and structural cleanliness up to `captured_at`; proceeds to Selector Gate 5 (`captured_at < onset`).

4.5 Five-Gate Selector Tie-Breaking & Dual-Hash Restore Verification

- **Deterministic Ranking & RPO Gap (`internal/selector/selector.go`):** Snapshots are sorted by `captured_at DESC`, then `file_count DESC`, then `snapshot_id DESC`. The first snapshot passing all 5 gates becomes `recommended_snapshot_id`; `rpo_gap_minutes` reports `(newest_snapshot.captured_at - recommended.captured_at)` in minutes so operators see exact data-age trade-offs.
- **Pre-Restore Live Gate + Post-Restore Dual Proof (`internal/restore/restore.go`):** Even if a client submits a manually chosen `snapshot_id`, `Execute()` calls `selector.EligibleNow()` (which re-runs `snapshots.Verify()` across every object on disk), writes each restored file via temp-file + `fsync` + atomic rename into `sandbox/restored/<target>-<runhex>/`, re-reads and SHA-256 hashes every output file, and finally re-verifies the source snapshot to record `source_unchanged = true`.

05 Deterministic Simulator (8 Families) & Measured Evaluation

`simulator/generate.py` and `simulator/scenarios.py` model a 120-file office workspace across `40` windows of `300s` each (3h20m simulated timeline, starting `2026-01-05T09:00:00Z`), capturing `13` periodic snapshots at windows `3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39`. Given `(family, seed)`, Python's seeded PRNG produces byte-identical mock files, event streams, and air-gapped `truth.json` files on any OS.

5.1 The 8 Deterministic Scenario Families

Scenario Family	Class	Active Windows	Synthetic Behavior & Expected Detector / Selector Verdict
`normal_office`	BENIGN	0 .. 39 (steady)	Routine low-rate edits/creates across docs, sheets, notes. **Expected:** 0 suspicious windows, no onset, newest snapshot (window 39) eligible & verified.
`benign_archive`	BENIGN	Window 28	15 compressed `.7z` files (~12.5% of tree) with high entropy (~7.9 bits/byte) created. **Expected:** Stays below mass-modification & 30% content thresholds; no false onset; newest snapshot eligible.
`benign_bulk_update`	BENIGN	Windows 28–29	60% of files updated in place with normal text entropy and unchanged extensions. **Expected:** Modified ratio rises alone without entropy/extension/delete signals (risk ≤ ~0.25 < 0.35); newest eligible.
`rapid_destructive`	ATTACK	Window 27	90% of files overwritten with uniform random bytes (H ≈ 7.95), inflated 1.35×, renamed to `.lockd`. **Expected:** Onset detected at window 27; snapshots ≥27 marked `affected`; clean snapshot @ window 24 restored.
`slow_distributed`	ATTACK	Windows 26–38	Low-rate 10% per-window high-entropy `.crypt` rewrites spread over 13 windows. **Expected:** Cumulative entropy shift + novel extension novelty crosses 0.35 at window 26; pre-onset snapshot @ window 24 chosen.
`extension_churn`	ATTACK	Windows 27–33	30% per-window mass renames to random novel extensions without relying on known ransomware suffixes. **Expected:** Rename rate + extension novelty trigger onset at window 27; snapshot @ window 24 selected.
`mixed_activity`	MIXED	Burst 27–28; Attack 30	Benign bulk update in windows 27–28 followed by 50% destructive `.lockd` rewrite in window 30. **Expected:** Benign burst skipped; onset pinned at window 30; fresher clean snapshot @ window 27 selected!
`pre_existing_contamination`	ABSTAIN	Window 0 (pre-snap)	90% destructive `.locked` rewrite occurs inside window 0 before the first snapshot (window 3). **Expected:** `BaselineContaminated()` voids certification; all snapshots `unknown`; selector returns `no_safe_candidate`.

5.2 Split Discipline & Measured Locked-Corpus Results

Per `simulator/seeds.json`, seeds are partitioned by whole `(family, seed)` runs into three disjoint splits: **Development** (seeds `1–6`, 60%), **Validation** (seeds `7–8`, 20%, used once to freeze `RiskThreshold = 0.35`), and **Locked** (seeds `9–10`, 20% = 16 full scenarios, untouched during tuning).

Locked-Corpus Metric (`results/eval-locked.json`)	Release Target	Measured Result	Comparison / Notes
Unsafe snapshot selections (`unsafe_selection_rate`)	**0 (Release-Blocking)**	**0 / 14 (0.0000)**	Zero contaminated snapshots ever recommended

Locked-Corpus Metric (`results/eval-locked.json`)	Release Target	Measured Result	Comparison / Notes
Naive "newest" baseline selector	Comparison baseline	**10 / 16 UNSAFE**	62.5% of runs would restore poisoned data
Naive "integrity-only" baseline selector	Comparison baseline	**10 / 16 UNSAFE**	Valid SHA-256 hashes fail to detect compromise
Recovery selection success (newest-eligible-clean)	Maximize clean RPO	**8 / 8 (100%)**	Always chose the freshest truly clean snapshot
Known-attack onset recall	≥ 0.90 (dev target)	**8 / 8 (1.0000)**	Detection delay: 2.12 – 4.80 min (5m window grid)
Benign-window false alert rate	≤ 0.05 (held-out)	**2 / 240 (0.0083)**	0.83% benign alert rate (6× below ceiling)
Pre-existing contamination abstention	Must abstain	**2 / 2 (100%)**	Both seeds returned `no_safe_candidate`
Verified isolated restores & source non-mutation	100% verified	**14 / 14 VERIFIED**	`all_restores_source_unchanged = true`

5.3 Measured Performance Benchmarks (1,000 Files × 32 KiB = 31.25 MiB)

Operation (Artifact in `results/`)	p50 Latency	p95 / Mean	Throughput	Measurement Notes (Host `Luna`, Go 1.26.1, Windows 11)
Snapshot capture (`snapshot-1000.json`)	1,196 ms	mean 6,593 ms	151.7 files/s (4.74 MB/s)	Iter 1 writes 31.25 MiB object store; iters 2–3 dedupe content-addressed objects
Verified restore (`restore-1000.json`)	10,451 ms	mean 10,736 ms	93.1 files/s (2.91 MB/s)	Includes pre-restore Verify(), atomic copy+rehash, and post-restore source Verify()
GET /recovery-candidates (`latency.json`)	~11 ms	p95 ~28 ms	—	20 samples against populated workload
GET /recovery-candidates @ c=20 (`throughput.json`)	—	—	36.6 req/s	Scales 6.8 → 27.4 → 33.8 → 36.6 req/s across concurrency 1 / 5 / 10 / 20

06 REST API Contract (`/api/v1`) & Phase M2 Web Console

All REST routes are specified in `api/openapi.yaml` under `/api/v1` (plus static web routes at `/`, `/landing/*`, and `/app/*`). Authentication uses HTTP `Authorization: Bearer <token>` with two distinct roles: **Operator** (read + state-changing mutations + audit/simulations) and **Read-Only** (safe inspection of workloads, events, snapshots, findings, candidates, evaluations, and metrics).

6.1 Complete HTTP Endpoint Matrix (21 Routes)

Method	Route	Min Role	Handler & Functional Contract
`GET`	`/` & `/landing/{file}`	PUBLIC	`landingPage` / `webUI`: Embedded cinematic landing page (whitelisted CSS/JS).
`GET`	`/app/{path...}`	PUBLIC*	`webUI`: Embedded React M2 SPA bundle with client-router `index.html` fallback (*API calls require token).
`GET`	`/api/v1/healthz`	PUBLIC	`handleHealth`: Returns `{"status":"ok","mode":"local-simulation","time":...}`.
`GET`	`/api/v1/metrics`	READ-ONLY	`metrics.Handler`: Prometheus text exposition (requests, captures, analyses, abstentions, restores, sims).
`GET`	`/api/v1/config`	READ-ONLY	`handleConfig`: Active detector thresholds, resource caps, CLI cheatsheet, and trust-boundary disclosures.
`POST`	`/api/v1/workloads`	OPERATOR	`handleCreateWorkload`: Registers sandbox-allowlisted workload root (`201` , `409` duplicate, `422` unsafe path).
`GET`	`/api/v1/workloads`	READ-ONLY	`handleListWorkloads`: Lists registered workloads newest-first.
`POST`	`/api/v1/workloads/{id}/events`	OPERATOR	`handleIngestEvents`: Validates & idempotently ingests batch (≤5,000 events; `422` on unsafe path).
`GET`	`/api/v1/workloads/{id}/events`	READ-ONLY	`handleListEvents`: Returns most recent ingested events (default 1,000, max 5,000).
`POST`	`/api/v1/workloads/{id}/snapshots`	OPERATOR	`handleCaptureSnapshot`: Serialized via `heavyGate`; walks root, writes objects, seals manifest (`201`).
`GET`	`/api/v1/workloads/{id}/snapshots`	READ-ONLY	`handleListSnapshots`: Lists snapshots newest-first enriched with current `assessment`.
`GET`	`/api/v1/snapshots/{id}`	READ-ONLY	`handleSnapshotDetail` (M2): Snapshot metadata, assessment, explanations, and paginated manifest file rows.
`POST`	`/api/v1/workloads/{id}/analyze`	OPERATOR	`handleAnalyze`: Runs `rules-v1` window scoring + snapshot integrity/exposure assessments (`200`).
`GET`	`/api/v1/workloads/{id}/findings`	READ-ONLY	`handleFindings`: Latest analysis run windows, per-signal contributions, onset, and affected file paths.
`GET`	`/api/v1/workloads/{id}/analysis-runs`	READ-ONLY	`handleAnalysisRuns` (M2): Historical analysis runs with parameters, window counts, and onset outcomes.
`GET`	`/api/v1/workloads/{id}/recovery-candidates`	READ-ONLY	`handleCandidates`: Ranked candidates with eligibility chips, RPO gap minutes, or `no_safe_candidate`.
`POST`	`/api/v1/workloads/{id}/restore`	OPERATOR	`handleRestore`: Serialized via `heavyGate`; requires `confirm:true`; executes verified sandbox restore (`201`).

Method	Route	Min Role	Handler & Functional Contract
`GET`	`/api/v1/workloads/{id}/recovery-runs`	READ-ONLY	`handleListRuns`: Lists historical restore runs for a workload.
`GET`	`/api/v1/recovery-runs/{id}`	READ-ONLY	`handleGetRun`: Recovery run metadata plus full per-file SHA-256 verification `report`.
`GET`	`/api/v1/evaluations`	READ-ONLY	`handleEvaluations` (M2): Serves measured `results/eval-locked.json` (path-traversal guarded).
`POST`	`/api/v1/simulate`	OPERATOR	`handleSimulateCreate` (M2): Launches whitelisted scenario job (`1..999` seed, single-concurrency `409 busy`).
`GET`	`/api/v1/simulations` & `/{id}`	OPERATOR	`handleSimulationList` / `Get` (M2): Live streaming driver log, phase progress, and truth verdict.
`GET`	`/api/v1/audit-events`	OPERATOR	`handleAudit`: Last 200 state-changing and rejected security actions (`403` for read-only tokens).

Uniform Error Envelope: Every non-2xx response returns JSON `{"code": "...", "message": "...", "details": [...], "request_id": "req-..."}` using 13 typed codes: `unauthenticated` (401), `forbidden` (403), `not_found` (404), `conflict` / `busy` / `not_eligible` (409), `confirm_required` / `invalid_input` / `no_telemetry` (400), `limit_exceeded` (413), `unsafe_path` / `root_unavailable` (422), and `internal` (500).

6.2 Phase M2 React + TypeScript Operator Console (All 15 PRD Views at `/app/`)

PRD Screen ID	React Component / Route	Implemented Capabilities & UX Safeguards
UI-01	`pages/Login.tsx` (`#/login`)	Masked bearer-token sign-in stored in `sessionStorage`; automatic Operator vs Read-Only role probing via `/config` + `/audit-events`.
UI-02	`pages/Overview.tsx` (`#/`)	6 live KPI cards, snapshot exposure dot-strips per workload, last verified restore banner, 30s pause-when-hidden auto-refresh, stale/skeleton states.
UI-03	`pages/Workloads.tsx` (`#/workloads`)	Searchable workload grid, summary stat strip, and operator **+ Register workload** modal (server-generated allowlisted roots only).
UI-04	`WorkloadDetail.tsx` (`overview`)	Header band with exposure/candidate/onset chips, operator **Capture snapshot** & **Analyze** actions, latest manifest summary, activity sparkline.
UI-05	`pages/Simulation.tsx` (`#/simulation`)	**Simulation Lab** control room: 8 tone-coded scenario tiles, seed stepper (1–999), 5-phase live progress bar, auto-scrolling driver log, verdict chips.
UI-06	`WorkloadDetail.tsx` (`events`)	Operation count pills (creates/modifies/deletes/renames), operation + path substring filters, old→new entropy and byte size columns.
UI-07 & 08	`WorkloadDetail.tsx` (`snapshots`)	Interactive SVG `SnapshotTimeline` + table + **Snapshot Detail Modal** showing explanation bullets and paginated SHA-256/entropy manifest files.
UI-09	`WorkloadDetail.tsx` (`findings`)	Interactive SVG `RiskChart` (shaded 24-window baseline, 0.35 threshold line, onset marker, clickable window contribution breakdown), affected files, and collapsible **Analysis history**.
UI-10	`WorkloadDetail.tsx` (`recovery`)	Newest-first candidate ranking, visual RPO `GapBar`, rejection reasons, and explicit red `no_safe_candidate` abstention banner (never falls back).
UI-11 & 12	`RestoreDialog` & `RunReportBody`	Operator approval modal (caveats, validated target label, mandatory confirmation checkbox) + clickable historical **Recovery runs** opening per-file SHA-256 verification reports.
UI-13	`pages/Evaluations.tsx` (`#/evaluations`)	Hero metric band + visual 3-policy **Selector Race** bar chart (`newest 10/16` vs `integrity-only 10/16` vs `CleanPoint 0/16` unsafe) from disk artifacts.

PRD Screen ID	React Component / Route	Implemented Capabilities & UX Safeguards
UI-14 & 15	`Audit.tsx` & `Settings.tsx`	Operator-only filterable audit feed (actions, rejected attempts, restores, simulations) + runtime config, CLI reference, and exact trust boundaries.

6.3 Middleware, RBAC & Simulation Lab Subprocess Architecture

Request Middleware & RBAC Chain — MIDDLEWARE.GO

- Every request passes through `WithRequestID`, generating `req-<16hex>`, recording Prometheus counters (`cleanpoint_http_requests_total`), and logging method, path, status, and duration.
- `Authenticate` extracts `Authorization: Bearer <token>` and compares against `OperatorToken` and `ReadOnlyToken` using constant-time `subtle.ConstantTimeCompare`.
- Failed auth (`401`) and role violations (`403`) write immutable rows to `audit_events` without ever logging token bytes.

Simulation Lab Orchestration — INTERNAL/API/M2.GO

- `POST /api/v1/simulate` validates the scenario name against the strict 8-family allowlist and bounds `seed ∈ [1, 999]`.
- `simRegistry` enforces single-concurrency execution (returning `409 busy` if a job is active) and retains the 32 most recent jobs in memory.
- `runSimulation` resolves `repoRoot()`, normalizes wildcard listeners (`0.0.0.0` → `127.0.0.1` via `localAPIURL()`), forwards `CLEANPOINT_SANDBOX_ROOT`, and streams stdout phases (`generating → ingesting → analyzing → selecting → restoring → complete`).

07 Security Architecture, Threat Model & 110-Case Test Matrix

7.1 In-Scope Threat Mitigations (Verified by Automated Suite)

Threat Vector	Architectural Mitigation	Verification Suite
Path Traversal & Backslash Smuggling (T01)	`security.ValidateRelativePath` rejects `..`, `.`, empty segments, leading `/`, drive letters, NUL bytes, DOS device names, and all `\` separators (`422 unsafe_path`).	`path_security_test.go` (14 cases)
Symlink Escape (T02)	Workload registration walks components via `os.Lstat` (`CheckNoSymlinks`); snapshot capture rejects any non-regular/symlink entry and invalidates the snapshot without reading outside `sandbox/roots/`.	`path_security_test.go` (symlink escape)
Unauthorized Mutation & Token Leakage	Constant-time bearer token auth; all POST routes require Operator role (`403` for Read-Only); `/audit-events` & `/simulate*` restricted to Operator; tokens stored `0600` and never logged or audited.	`api_auth_test.go` + `m2_api_test.go`
Bit-Rot / Tampered Snapshot Objects (T03/T04)	`snapshots.Manager.Verify()` recomputes canonical `ManifestHash` from SQL rows and re-hashes every stored object file during analysis and again immediately prior to restore.	`snapshot_integrity_test.go` (11 cases)
Valid-Hash Contaminated Backup (T05)	Behavioral onset gate (`captured_at < onset`) + manifest content check (`≥30%` novel ext @ entropy `≥7.0`) + baseline contamination check prevent restoring intact-but-encrypted snapshots.	`e2e_scenarios_test.go` + `detection_test.go`
All Backups Compromised (T06)	Selector returns `status: "no_safe_candidate"` with `recommended_snapshot_id: null` and deduplicated reasons; restore endpoint rejects ineligible snapshot IDs with `409`.	`recovery_eligibility_test.go` (13 cases)
Accidental Live-Source Overwrite	Client supplies only a `[a-z0-9-]` label; server generates unique `sandbox/restored/<label>-<runhex>/` destination and re-verifies source snapshot post-copy (`source_unchanged == true`).	`restore_test.go` (11 cases)
Capture / Restore Race Conditions (T10)	Single-writer SQLite connection + `heavyGate` non-blocking mutex serialize concurrent captures/restores cleanly with `409 busy`.	Integration concurrency & gate tests

7.2 Explicit Scope Boundaries & Known Limitations (`docs/limitations.md`)

> **Architectural Honesty Boundaries**
> **1. Logical Immutability vs Hardware WORM:** Sealed snapshots cannot be modified or deleted via HTTP APIs or SQLite cascades (`ON DELETE RESTRICT`), but a local OS administrator can still alter `sandbox/objects/` on disk (which `Verify()` detects and rejects). **2. Synthetic PRNG Corpus:** Scenarios model statistical file-mutation shapes rather than kernel/process telemetry or live malware evasion. **3. Coarse 5-Minute Window Grid:** Onset timestamps align to `300s` window boundaries, trading up to one window of RPO margin for zero contaminated selections.

7.3 Automated Verification Matrix (110 Shipped Cases vs 92 Minimum)

Test Suite	Test File	Mandated Min	Shipped & Passing
1. Path & Filesystem Security	`tests/security/path_security_test.go`	12	**14**
2. Snapshot Capture & Integrity	`tests/integration/snapshot_integrity_test.go`	10	**11**

Test Suite	Test File	Mandated Min	Shipped & Passing
3. Idempotent Event Ingestion	`tests/integration/snapshot_integrity_test.go`	8	**10**
4. Window Feature Extraction	`tests/recovery/feature_extraction_test.go`	12	**13**
5. Anomaly Detection (`rules-v1`)	`tests/recovery/detection_test.go`	12	**14**
6. Recovery Candidate Eligibility	`tests/recovery/recovery_eligibility_test.go`	12	**13**
7. Verified Restore Correctness	`tests/recovery/restore_test.go`	10	**11**
8. API Authentication & RBAC	`tests/integration/api_auth_test.go`	8	**11**
9. End-to-End Scenario Matrix	`tests/integration/e2e_scenarios_test.go`	8	**8**
10. Phase M2 Web UI & Support Routes	`tests/integration/m2_api_test.go`	—	**6**
Total Automated Cases	`go test ./tests/... -v -count=1`	**92**	**110 PASS**

08 End-to-End Working Runbook (Build, Test, Demo & Docker)

8.1 Prerequisites

- **Go 1.26+** (pure-Go SQLite; **no CGO / C compiler required**) & **Python 3.11+** (standard library only; zero `pip` packages).
- **Node.js 20+ / npm** (optional — required only when editing `web/console/src/` ; fresh clones embed prebuilt `web/console-dist/`).

8.2 Windows PowerShell Workflow (Native One-Command Scripts)

```
# 1. Build M2 React console (if npm present) + all 3 Go binaries into .\bin + go vet
.\scripts\build.ps1

# 2. Run go vet + TypeScript strict check + all 110 Go tests + Python determinism check
.\scripts\test.ps1

# 3. Boot local server on http://127.0.0.1:8099 and run the 3-scene guided demonstration
.\scripts\demo.ps1

# 4. Inspect generated credentials for the Web Console (http://127.0.0.1:8099/app/)
Get-Content .\sandbox\.tokens
```

8.3 Linux / macOS / Git-Bash & Direct CLI Workflow

```
# Build & test via Makefile or Bash scripts
make build && make test          # or: scripts/bootstrap.sh && scripts/test.sh
make demo                        # starts server on :8099 and runs 3-scene demo

# Direct CLI operations against a live server (CLEANPOINT_API=http://127.0.0.1:8099)
./bin/cleanpoint.exe serve
./bin/cleanpoint.exe simulate --scenario rapid_destructive --seed 1 --api http://127.0.0.1:8099
./bin/cleanpoint.exe evaluate --scenario locked --seeds 9,10 --json --output results/eval-locked.json --api
http://127.0.0.1:8099
./bin/cleanpoint.exe benchmark snapshots --files 1000 --file-size-kb 32 --output results/snapshot-1000.json --
api http://127.0.0.1:8099
./bin/cleanpoint.exe benchmark restore   --files 1000 --file-size-kb 32 --output results/restore-1000.json  --
api http://127.0.0.1:8099
```

8.4 Optional Multi-Stage Container Workflow (Loopback-Bound)

```
# Multi-stage build: Node 22 (web/console) -> Go 1.26 (3 static binaries) -> Python 3.12-slim
docker compose up --build -d
# Landing page: http://127.0.0.1:8080/   |   M2 Operator Console: http://127.0.0.1:8080/app/
```

8.5 Complete Runtime Environment Variables Reference (`internal/config/config.go`)

Environment Variable	Default Value	Runtime Behavior & Security Constraints
`CLEANPOINT_ADDR`	`127.0.0.1:8080`	HTTP bind address. Loopback by default; non-loopback binds emit an explicit warning banner.
`CLEANPOINT_SANDBOX_ROOT`	`./sandbox`	Root directory containing `roots/`, `objects/`, `restored/`, `cleanpoint.db`, and `.tokens`.
`CLEANPOINT_DB_PATH`	`<sandbox>/cleanpoint.db`	SQLite WAL database file path (auto-migrated on boot via `migrations/001_init.sql`).
`CLEANPOINT_OPERATOR_TOKEN`	Auto-generated (`.tokens`)	Bearer token granting full read + state-changing mutation + audit + Simulation Lab privileges.
`CLEANPOINT_READONLY_TOKEN`	Auto-generated (`.tokens`)	Bearer token restricted to safe `GET` inspection endpoints; rejected with `403` on mutations.
`CLEANPOINT_PYTHON`	`python` (or `python3`)	Python 3.11+ interpreter invoked by `cleanpoint` CLI and M2 Simulation Lab jobs.
`CLEANPOINT_WINDOW_SECONDS`	`300` (5 minutes)	Non-overlapping telemetry window width used by `rules-v1` feature extraction.
`CLEANPOINT_BASELINE_WINDOWS`	`24` (2 hours)	Number of initial windows used for Median/MAD calibration before onset evaluation begins.
`CLEANPOINT_RISK_THRESHOLD`	`0.35`	Validation-calibrated suspicion cutoff in `[0, 1]` separating normal vs suspicious windows.
`CLEANPOINT_MAX_SNAPSHOT_FILES`	`5000`	Maximum regular files allowed per captured snapshot (returns `413 limit_exceeded` above cap).
`CLEANPOINT_MAX_SNAPSHOT_BYTES`	`104857600` (100 MiB)	Maximum cumulative bytes allowed per captured snapshot to prevent local disk exhaustion.